

Skills от А до Я

Как научить AI-агента
делать что угодно

Создание, тестирование и автоматизация
навыков для AI-агентов

ОЛЕГ ЛАВРЕНТЬЕВ · А П Р Е Л Ь 2 0 2 6

Зачем этот гайд

Скиллы — самый недооценённый инструмент в AI-автоматизации. Все говорят про [промпы](#), [MCP-серверы](#), мультиагентные архитектуры — а потом один [Markdown](#)-файл решает задачу, на которую раньше уходила команда из 8 агентов.

Этот гайд — не теория из документации. Здесь конкретные архитектуры, реальные кейсы и рабочие паттерны. От простого текстового скилла на 30 строк до production-системы с 15 [скриптами](#) и 10 reference-файлами.

Если ты никогда не создавал скилл — после этого гайда создашь.

Если уже создавал — узнаешь как сделать его стабильнее, экономичнее и автономнее.

Встретил незнакомый термин? Все определения — в [Словаре терминов](#) в конце гайда.

Оглавление

Часть I — Как мы к этому пришли

1. 8 агентов, 8 точек отказа	3
2. Один Skill, который заменил их всех	4

Часть II — Что такое скилл и как он устроен

3. Скилл — это просто файл	5
4. Три уровня сложности: от рецепта до конвейера	7
5. Почему контекст — это всё	9
6. Где живут скиллы	11

Часть III — Создаём свой скилл

7. Три способа создать скилл	14
8. Практика: создаём скиллы	15
9. Description: 400 символов, которые решают всё	28
10. Инструкции: как разговаривать с моделью	29
11. Тестируй, ломай, чини	33

Часть IV — Как мы сделали devops-ак

12. Файловая структура	36
13. SKILL.md строка за строкой	37
14. Скрипты: когда модели нужны руки	39

Часть V — На продакшн

15. От прототипа к стабильной системе	40
16. Автоматизация: чтобы скилл работал без тебя	41
17. Скиллы на стероидах: воркфлоу, цепочки, оркестрация	42
18. Безопасность: чужие скиллы и свои грабли	44

Справочник

Шаблон SKILL.md	46
Чеклист готовности	47
Словарь терминов	48
Ссылки и ресурсы	51

Часть I — Как мы к этому пришли

1. 8 агентов, 8 точек отказа

Полгода назад перед нами стояла задача: автоматизировать настройку серверов. Купил [VPS](#), нужно поставить [Docker](#), Caddy, n8n, Supabase, настроить [DNS](#), [SSL](#), [файрвол](#), сгенерировать отчёт с логинами.

Единственный инструмент управления моделью в то время — промпт. Хочешь сложную автоматизацию — пиши несколько агентов, каждый со своим промптом, и координируй их вручную.

Мы так и сделали. [Собрали систему из 8 DevOps-агентов в Kilo Code](#):

Агент	Задача
base-server-setup	Базовая настройка сервера, сбор данных
n8n-deployment	Деплой n8n + PostgreSQL
supabase-deployment	Деплой Supabase
cloudflare-dns-manager	DNS-записи в Cloudflare
nginx-proxy-configurator	SSL-проксирование для всех сервисов
system-validator	Проверка что всё работает
final-report-agent	Финальный отчёт для клиента
devops-chain-orchestrator	Координирует остальных семерых

8 промптов. 8 конфигураций. 8 точек, где что-то может пойти не так. И это только для **двух сервисов** — n8n и Supabase.

Что пошло не так:

- **Контекст терялся.** [Оркестратор](#) передавал задачу DNS-агенту, тот делал записи — но когда результат возвращался обратно, оркестратор уже "забыл" часть предыдущих шагов. [Контекстное окно](#) не резиновое.
- **Координация жрала токены.** Половина [токенов](#) уходила не на работу, а на общение между агентами: "передай DNS-агенту", "получи результат от DNS-агента", "проверь что DNS-агент закончил".
- **Хрупкость.** Один агент упал → цепочка сломалась. Перезапустить с середины невозможно — нужно начинать сначала.

- **Узкий функционал.** Хочешь добавить бэкапы? Нужен 9-й агент. Мониторинг? 10-й. Каждая новая задача = новый промпт + обновление оркестратора.

Система работала. Но с болью. И масштабировать её было невозможно.

2. Один Skill, который заменил их всех

Потом появились скиллы ([Agent Skills](#)) — стандартный формат, который приняли Anthropic, Google, OpenAI и Perplexity. И мы переписали всю эту систему в один скилл: `devops-ak`.

Было: 8 агентов

Умели: поставить n8n + Supabase. Всё.

Стало: 1 скилл

Умеет: поставить ЛЮБОЙ софт + бэкапы + мониторинг + обновления + деплой проектов с [GitHub](#) + DNS/SSL + генерация PDF-отчётов + и вообще что угодно на сервере

Функционал вырос в разы. Сложность управления упала до нуля. Одна команда `/devops-ak` — и агент знает что делать.

Как именно это устроено — разберём по косточкам в [Части IV](#). Но сначала — что вообще такое скилл и как устроен формат.

Часть II — Что такое скилл и как он устроен

3. Скилл — это просто файл

Скилл — это текстовый файл с пошаговыми инструкциями. Буквально Markdown-документ, в котором написано: "когда попросят X — делай Y, потом Z, проверь результат".

Как рецепт: шаг 1, шаг 2, шаг 3. Только вместо блюда на выходе — настроенный сервер, составленный отчёт, обработанная заявка.

Минимальный скилл — одна папка с одним файлом:

```
my-skill/  
SKILL.md ← весь скилл здесь
```

SKILL.md состоит из двух частей:

```
---                                ← YAML-метаданные (имя + описание)  
name: my-skill  
description: "Что делает. Когда использовать. Ключевые слова."  
---                                ← конец метаданных  
  
# Название                        ← дальше Markdown-инструкции  
  
## Когда активировать  
## Пошаговые инструкции  
## Примеры результата  
## Правила
```

Отличие от промпта:

	Промпт	Skill
Формат	Текст в чате	Структурированный файл
Контекст	Всё в одном сообщении	Подгружается поэтапно
Повторяемость	Копируешь вручную	Вызывается одной командой
Переносимость	Привязан к платформе	Работает в Claude Code, Gemini CLI, Codex CLI, Perplexity
Масштаб	Один запрос	Многошаговые воркфлоу с валидацией

Промпт загружает все инструкции разом. Скилл загружает их поэтапно — сначала описание ("нужен ли я?"), потом инструкции ("что делать?"), потом детали по конкретной подзадаче.

Представь: ты открыл книгу на 300 страниц и пытаешься удержать в голове всё одновременно. Или ты открыл оглавление, нашёл нужную главу и читаешь только её. Скилл — это второй вариант.

Скилл — это не одно действие. Один скилл может объединять десятки операций в один процесс, в том числе обращаясь к разным внешним сервисам и **API**: забрать данные из базы (Supabase), найти контекст в базе знаний (LightRAG), сгенерировать PDF-отчёт, отправить уведомление в Telegram — и всё это по одной команде. Каждый шаг прописан, порядок зафиксирован, агент идёт по инструкции от начала до конца.

Скилл не делает модель умнее. Модель и так знает, как писать **Docker Compose**, составлять контент-план или анализировать договор. Скилл говорит: "делай это в ТАКОМ порядке, с ТАКИМИ параметрами, проверяй ТАК". Направляет, а не обучает.

Когда скилл нужен, а когда нет:

- Делаешь что-то чаще 3 раз → пиши скилл
- Делаешь один раз → просто напиши в чат
- Задача требует стабильного, повторяемого результата → скилл
- Задача разовая и творческая → обычный промпт

Где скиллы работают:

Единый стандарт (Agent Skills). Один и тот же SKILL.md работает в Claude Code, Gemini CLI, Codex CLI, Perplexity Computer, **OpenClaw** и 20+ других платформах. Написал один раз — запускай где угодно.

4. Три уровня сложности: от рецепта до конвейера

Скиллы масштабируются. Начни с простого — усложняй когда нужно.

Уровень 1: Просто SKILL.md

Один файл, 20-50 строк. Подходит для 80% задач: консультации, генерация контента, анализ, подготовка документов.

```
---
name: nutrition-advisor
description: "Составление рациона питания и рекомендации
  по нутрициологии. Используй когда: питание, диета, рацион,
  калории, БЖУ, что поесть, здоровое питание, nutrition."
---

# Консультант по питанию

Отвечай на русском.

## Алгоритм
1. Спроси цель: похудение / набор массы / поддержание / здоровье
2. Спроси ограничения: аллергии, непереносимости, вегетарианство
3. Спроси текущий режим: сколько приёмов пищи, что обычно ешь
4. Рассчитай норму КБЖУ по формуле Миффлина-Сан Жеора
5. Предложи 3 варианта дневного рациона

## Правила
- НЕ ставь диагнозы
- При серьёзных проблемах → направь к врачу-диетологу
```

30 строк. Ноль скриптов. Работает.

Уровень 2: SKILL.md + references/

Когда информации много — выноси детали в отдельные файлы. SKILL.md становится **маршрутизатором**: определяет тип запроса и отправляет к нужному reference.

```
content-plan/
├── SKILL.md           ← маршрутизатор (50 строк)
└── references/
```

```
|— formats-guide.md      ← форматы: пост, сторис, карусель, тредс
|— tone-checklist.md     ← чеклист тона и стоп-слов
|— promo-templates.md    ← шаблоны прогревающих постов
```

SKILL.md: "Если нужен контент-план → алгоритм ниже. Если нужен конкретный формат поста → см. [references/formats-guide.md](#) . Если нужна проверка тона → см. [references/tone-checklist.md](#) ."

Модель загружает только нужный reference, а не все три. Экономия контекста, точнее результат.

Правило: если SKILL.md перевалил за ~3-4 страницы (~5000 токенов) — пора выносить в references.

Уровень 3: SKILL.md + references/ + scripts/

Когда нужна предсказуемость и повторяемость: установка софта, генерация документов, обращение к внешним API.

```
logistics-onboarding/
|— SKILL.md              ← воркфлоу онбординга
|— references/
| |— documents-checklist.md ← список документов по категориям
| |— troubleshooting.md    ← частые проблемы
|— scripts/
| |— create-profile.sh      ← создание профиля в системе
| |— check-fleet-api.sh     ← проверка статуса в API перевозчика
| |— send-welcome.sh       ← отправка welcome-сообщения
```

Скрипты делают то, что модель каждый раз делала бы чуть по-разному: обращение к API, генерация [credentials](#), создание записей в базе. Один и тот же скрипт, одни и те же параметры — протестировано 50 раз.

Есть ещё [assets/](#) — шаблоны для результата. Отличие от references: references — инструкции "как делать" (для модели), assets — заготовки "из чего делать" (шаблоны отчётов, конфиги, которые модель заполняет данными).

Начинай с уровня 1. Большинству задач хватает одного SKILL.md. Усложняй только когда реально упрёшься в ограничения.

5. Почему контекст — это всё

Весь дизайн скиллов построен на одной идее: **контекстное окно модели — это её оперативная память, и она ограничена.**

Вот что происходит когда модель работает:

Контекстное окно (допустим, 200К токенов):

Что занимает место	Токены
Системный промпт + конфиг платформы	~5 000
SKILL.md (после активации)	~2 000
Reference-файл (если нужен)	~500
История диалога с пользователем	~10 000
Результаты команд и работы	~50 000
СВОБОДНОЕ МЕСТО для работы	~132 500

Если запихнуть все инструкции сразу — токены потрачены впустую. На длинной сессии это реально упирается в потолок.

Три принципа, которые мы вывели:

Принцип 1: Загружай только то, что нужно прямо сейчас.

SKILL.md = маршрутизатор (~2000 токенов). References = детали (~500 токенов каждый). Загружаются по одному. Модель видит только те инструкции, которые ей нужны на текущем шаге.

Принцип 2: Не объясняй модели то, что она уже знает.

Модель знает, что такое Docker, как написать compose-файл, как составить SWOT-анализ. Не трать токены на учебник. Пиши только СВОЮ специфику: порядок действий, конкретные параметры, формат результата, бизнес-правила.

Плохо (модель и так это знает):

Docker Compose — это инструмент для определения и запуска многоконтейнерных Docker-приложений...

Хорошо (конкретика, которую модель не знает):

Порядок установки:

1. setup-server.sh → Docker, UFW
2. install-caddy.sh → reverse proxy, сеть infra

- 3. cloudflare-dns.sh → A-записи
- 4. LightRAG → ПОСЛЕДНИМ

Принцип 3: Каждый токен должен оправдывать своё присутствие.

Если убрать фрагмент инструкции и результат не ухудшится — убирай. Чем меньше шума в контексте, тем точнее модель выполняет задачу.

Про инструменты: меньше = лучше

Если скилл работает в среде с кучей **МСП-серверов** и инструментов — модель тратит токены на выбор между ними. Несколько правил:

- **Минимальный набор.** Не подключай инструменты, которые скиллу не нужны. Задача про генерацию контента — не нужны инструменты для работы с базами данных.
- **Понятные описания.** Модель выбирает инструмент по его описанию. Плохо: `tool_1`. Хорошо: `extract_text_from_pdf`.
- **Минимальное перекрытие.** Два инструмента с похожими описаниями путают модель — она не знает какой выбрать.
- **Компактные результаты.** Инструменты должны возвращать структурированный ответ, а не сырой дамп на 500 строк.

Что происходит когда контекст забивается

Мы это прочувствовали на devops-ак. Полная установка сервера — это 6 фаз, 6-8 сервисов, десятки **bash**-команд. К финальной фазе контекст забит: SKILL.md + несколько references + результаты 30+ команд + весь диалог. Начинается деградация:

- Модель "забывает" ранние инструкции. Правило "NEVER delete without confirmation" утонуло в сотнях строк вывода.
- Модель пропускает шаги. Из 8 скриптов помнит что запустила 5, хотя запустила 7.
- Формат ответов плывёт. В начале — структурированный вывод, к концу — поток текста.

Как мы это решили:

1. **TodoWrite для трекинга фаз.** Модель создаёт чеклист в начале и отмечает каждую фазу. Даже когда инструкции "тонут" — чеклист всегда на виду.
2. **Минимизация вывода.** Скрипты пишут **JSON**-лог в файл, а в stdout — только статус: "OK: n8n installed" или "ERROR: port busy". Не дампят весь compose в контекст.

3. **Жёсткие правила в конце файла.** Секция Rules стоит в конце SKILL.md — модель видит её последней, и она лучше удерживается в "свежей памяти".

6. Где живут скиллы

Глобальные и проектные

Глобальные — доступны всегда, из любого проекта:

```
~/ .claude/skills/my-skill/SKILL.md
```

Для универсальных скиллов: ответы на отзывы, генерация КП, конспекты встреч — всё что нужно независимо от того, в какой папке ты сейчас работаешь. Открыл терминал где угодно — скилл доступен.

Проектные — только в рамках конкретного проекта:

```
.claude/skills/my-skill/SKILL.md
```

Для специфичных задач: деплой именно этого сервера с его конфигурацией, работа с этой конкретной базой данных, сборка именно этого проекта. Зашёл в другую папку — скилла нет, и это правильно, потому что он там не нужен.

Когда какой выбирать? Если скилл привязан к конкретному проекту — проектный. Если пригодится везде — глобальный. Не уверен — начни с глобального, потом перенесёшь если нужно.

Агент автоматически сканирует обе директории при запуске.

Как скилл запускается

Способ 1: Автоматически — по словам в чате. Модель видит описания всех скиллов. Ты пишешь "подними n8n на сервере" → модель находит скилл, в description которого есть "подними", "n8n", "сервер" → активирует его. Ты даже не знаешь, что скилл сработал — просто получаешь качественный результат.

Плюсы: удобно, не нужно помнить команды.

Минусы: не 100% надёжно. Короткие или нестандартные запросы могут не **триггернуть**. Иногда модель выбирает не тот скилл.

Способ 2: Слэш-команда — явный запуск. В чате набираешь `/` — появляется список всех доступных скиллов. Выбираешь нужный или дописываешь имя: `/devops-ak`. Скилл гарантированно активируется. Без угадывания.

Плюсы: 100% надёжность.

Минусы: нужно помнить название (но автодополнение помогает).

Что выбрать? Для повседневной работы — пиши обычным языком, пусть модель сама находит нужный скилл. Для критичных задач и автоматизации (`scheduled tasks`, `крон`) — слэш-команда. Когда ставишь скилл на расписание "каждые 6 часов проверяй сервер" — нужна гарантия что скилл сработает, а не "модель решила что это не про неё".

CLAUDE.md vs SKILL.md — не путай

	CLAUDE.md	SKILL.md
Загружается	Всегда, каждая сессия	Только при активации
Для чего	Правила проекта, стиль кода	Конкретная задача
Где	Корень проекта	<code>.claude/skills/имя/</code>
Аналогия	Правила дома	Инструкция к прибору

CLAUDE.md — "всегда помни". **SKILL.md** — "когда попросят, делай так". Не дублируй.

Один формат — все платформы

Стандарт Agent Skills работает везде. Написал SKILL.md — он работает в:

Платформа	Путь к скиллам
Claude Code / Desktop	<code>.claude/skills/</code> или <code>~/.claude/skills/</code>
Gemini CLI	<code>.gemini/skills/</code> или <code>~/.gemini/skills/</code>
Codex CLI	<code>.claude/skills/</code>
Perplexity Computer	Загрузка через страницу управления
OpenClaw	<code>.openclaw/workspace/skills/</code>
Cursor	<code>.cursor/skills/</code>
Windsurf	<code>.windsurf/skills/</code>
Kilo Code	<code>.kilo/skills/</code>

OpenClaw — отдельная история: агент работает 24/7 на сервере, общается через Telegram. Скиллы привязаны к конкретному агенту (у каждого бота свой workspace). Формат тот же — написал в Claude Code, перенёс в OpenClaw без изменений.

Часть III — Создаём свой скилл

7. Три способа создать скилл

Способ 1: Из текущего чата (самый быстрый)

Ты уже работаешь с агентом — составляете контент-план, настраиваете процесс, анализируете данные. Результат нравится. Просто скажи:

"Оформи то, что мы сейчас делали, в виде скилла.
Чтобы я мог повторить это одной командой."

Агент соберёт контекст диалога, выделит пошаговый алгоритм, создаст SKILL.md и положит в `.claude/skills/`. Готово — в следующий раз вызываешь одной командой.

Способ 2: Через `/skill-creator` (с тестированием)

В Claude Code есть встроенный скилл `/skill-creator`. Он не просто создаёт скилл — он его тестирует:

1. Генерирует тест-кейсы (запросы, которые должны и не должны триггерить)
2. Делает 60/40 `train/test split`
3. Прогоняет каждый запрос 3 раза
4. Итерирует до 5 раз, оптимизируя `description` по `test score`
5. Открывает HTML-отчёт с результатами

По сути — unit-тесты для скилла. Описываешь что нужно, `skill-creator` создаёт, тестирует и дорабатывает. Полезно для скиллов, которые должны точно триггериться на нужные запросы.

Способ 3: Вручную (для понимания)

Можно и руками — `mkdir -p .claude/skills/my-skill && vim SKILL.md`. Но зачем, если есть `skill-creator` и генерация из чата? Ручное создание полезно для понимания формата и для тонкой настройки, но стартовать лучше через первые два способа.

8. Практика: создаём скиллы

Ответы на отзывы — простой скилл через /skill-creator

Теория — это хорошо. Но лучше один раз увидеть. Разберём создание скилла от и до — от запуска `/skill-creator` до готового результата.

Задача: Ты владелец кофейни (салона, клиники, автосервиса — неважно). Каждый день приходят отзывы в 2ГИС, Яндекс.Картах, Google. На позитивные нужно поблагодарить, на негативные — ответить вежливо и конструктивно. Руками это 15-20 минут в день. Автоматизируем.

Шаг 1: Запускаешь skill-creator и описываешь задачу.

`/skill-creator`

Создай скилл для генерации ответов на отзывы клиентов.

Как это работает: я вставляю текст отзыва — получаю готовый ответ.

Тон: профессиональный, тёплый, без канцелярита.

Если отзыв негативный — признаём проблему, предлагаем решение.

Примеры:

Позитивный отзыв: "Отличный кофе, приду ещё!" →

Ответ: "Спасибо за тёплые слова! Будем рады видеть вас снова 😊"

Негативный: "Ждал заказ 40 минут, это неприемлемо" →

Ответ: "Приносим извинения за ожидание. Мы разобрались в ситуации — в тот день была пиковая нагрузка. Приходите снова, угостим кофе за наш счёт как извинение."

Шаг 2: skill-creator задаёт уточняющие вопросы — для какого бизнеса, нужны ли тест-кейсы, какие площадки (2ГИС, Яндекс.Карты, Google). Отвечаешь — он уточняет задачу.

Шаг 3: skill-creator создаёт SKILL.md, генерирует тест-кейсы и прогоняет их. Ты видишь: какие запросы **триггернули** скилл, какие нет. Если description недостаточно точный — skill-creator итерирует до 5 раз, оптимизируя формулировку.

Шаг 4: Открывается HTML-отчёт с результатами. Ты видишь выходы по каждому тест-кейсу, можешь оставить фидбек: "тут слишком формально", "тут забыл предложить компенсацию". skill-creator дорабатывает.

Результат — готовый скилл:

name: review-reply

description: "Генерация ответов на отзывы клиентов для 2ГИС, Яндекс.Карт, Google. Позитивные, негативные, нейтральные. Профессиональный тон.

Триггеры: отзыв, ответ на отзыв, review reply, клиент написал, ответить на отзыв, 2ГИС, Яндекс.Карты."

Ответы на отзывы клиентов

Отвечай на русском.

When activated

1. Пользователь вставил текст отзыва клиента
2. Нужно сгенерировать профессиональный ответ на отзыв
3. Нужно адаптировать тон под тип отзыва (позитив / негатив / нейтрал)

Instructions

Step 1: Определи тип отзыва

- Позитивный — клиент доволен, хвалит
- Негативный — жалоба, недовольство, проблема
- Нейтральный — констатация факта, без эмоций

Step 2: Сгенерируй ответ по шаблону

****Позитивный:****

- Поблагодари конкретно (не "спасибо за отзыв", а за что именно)
- Подчеркни что команда старается
- Пригласи прийти снова

****Негативный:****

- Признай проблему (не отрицай, не оправдывайся)
- Объясни что сделано / будет сделано
- Предложи компенсацию или решение
- Дай контакт для связи

****Нейтральный:****

- Поблагодари за обратную связь
- Уточни если можно что-то улучшить

Step 3: Проверь перед выдачей

- Тон: тёплый, профессиональный, без канцелярита
- Нет шаблонных фраз ("Уважаемый клиент", "Благодарим за обращение")
- Длина: 2-4 предложения для позитивных, 3-6 для негативных
- Если в отзыве имя клиента — обратиться по имени

Rules

- НЕ используй казённый язык
- НЕ спорь с клиентом, даже если он неправ
- НЕ обещай то, что бизнес не может выполнить
- Каждый ответ уникальный — не копируй предыдущие

Готово. Файл `~/ .claude/skills/review-reply/SKILL.md` создан, протестирован, description оптимизирован. Теперь пишешь в чат:

Ответь на отзыв: "Были вчера с друзьями. Капучино — огонь, но десерты так себе, суховатые какие-то."

И получаешь:

Спасибо, что поделились впечатлениями! Рады, что капучино понравился — бариста старается 😊 А по десертам приняли к сведению, спасибо.

20 строк SKILL.md. Пара минут через `/skill-creator`. Вместо 15 минут в день на ответы — 15 секунд. И главное — skill-creator не просто создал скилл, а **протестировал** его: проверил что он триггерится на нужные запросы, что ответы соответствуют тону, что description работает. Ты получил скилл, который уже прошёл через unit-тесты.

Обрати внимание: этот скилл — чистый текст. Никаких API, скриптов, интеграций. Ты не программист? Не важно. Ты описал задачу человеческим языком, skill-creator всё сделал за тебя. Скопировал ответ, вставил в 2ГИС — готово. Даже такие простые скиллы экономят часы в неделю. Не нужно

сразу строить ракету — начни с того, что уже болит. Усложнить всегда успеешь.

Коммерческое предложение — скилл с reference-файлами

Предыдущий скилл — один файл, 20 строк. Теперь задача посложнее: КП должно подтягивать данные из шаблона, прайс-листа, условий работы. Всё это не влезет в SKILL.md — и не должно. Для этого есть reference-файлы.

Задача: Ты руководитель отдела продаж. Менеджеры каждый день составляют коммерческие предложения — копируют шаблон в Word, вручную подставляют данные, ищут цены в таблице. На одно КП — 30-40 минут. Автоматизируем.

Что пишешь в skill-creator:

У тебя уже есть файлы — шаблон КП в Google Docs или Word, прайс-лист в таблице, условия работы где-то в договоре. Ты просто говоришь skill-creator где они лежат:

```
/skill-creator
```

Создай скилл для генерации коммерческих предложений.

Как работает: я указываю название компании клиента, какую услугу предлагаем и примерный бюджет. Скилл берёт наш шаблон, подставляет данные и генерирует готовое КП.

Вот мои файлы:

- Шаблон КП: ~/Documents/Работа/шаблон-кп.docx
- Прайс-лист: ~/Documents/Работа/прайс-2026.xlsx
- Условия работы: ~/Documents/Работа/условия-договор.pdf

Перенеси их в скилл как reference-файлы, сконвертируй в markdown чтобы агент мог их читать.

skill-creator сам откроет файлы, сконвертирует в markdown, положит в `references/` и создаст SKILL.md, который на них ссылается. Тебе не нужно вручную ничего переносить и форматировать — всё автоматически.

```
commercial-proposal/  
├── SKILL.md  
└── references/
```

— kp-template.md	← шаблон КП
— price-list.md	← прайс на услуги
— terms.md	← условия работы

Ключевое отличие от первого примера: SKILL.md не содержит шаблон и цены — он **ссылается** на reference-файлы. Это **маршрутизация**: скилл знает какой файл подгрузить в зависимости от запроса.

Результат — SKILL.md:

```

---
name: commercial-proposal
description: "Генерация коммерческих предложений по шаблону компании.
Подставляет данные клиента, рассчитывает стоимость по прайсу,
добавляет условия работы. Триггеры: КП, коммерческое предложение,
commercial proposal, составь КП, предложение для клиента,
proposal, сделай предложение."
---

# Генерация коммерческого предложения

Отвечай на русском.

## When activated
1. Нужно составить коммерческое предложение для клиента
2. Нужно рассчитать стоимость услуг по прайсу
3. Нужно обновить существующее КП с новыми данными

## Route by request type
1. Новое КП → алгоритм ниже
2. Вопрос по ценам → see references/price-list.md
3. Вопрос по условиям → see references/terms.md

## Instructions

### Step 1: Собери данные о клиенте
Спроси (если не указано):
- Название компании клиента
- Контактное лицо
- Какая услуга / набор услуг нужен
- Примерный объём (если применимо)

### Step 2: Рассчитай стоимость
- Открой references/price-list.md

```

- Найди запрошенные услуги
- Рассчитай итоговую сумму
- Если бюджет клиента указан — предложи оптимальную комбинацию

Step 3: Собери КП

- Возьми структуру из references/kp-template.md
- Подставь данные клиента
- Вставь рассчитанные цены
- Добавь условия из references/terms.md
- Укажи дату и срок действия (14 дней)

Step 4: Проверь

- Все поля заполнены
- Цены соответствуют прайсу
- Нет ошибок в названии компании
- Итоговая сумма посчитана правильно

Rules

- Цены ТОЛЬКО из references/price-list.md — не выдумывай
- Шаблон ТОЛЬКО из references/kp-template.md — не меняй структуру
- Условия ТОЛЬКО из references/terms.md — не обещай другое
- Если услуги нет в прайсе — скажи об этом, не подставляй цену

Как работает:

Составь КП для компании "Рассвет". Им нужна разработка лендинга и настройка контекстной рекламы. Бюджет — до 300 тысяч.

Скилл подгружает прайс → находит цены на лендинг и рекламу → проверяет что вписывается в 300к → подставляет в шаблон → выдаёт готовое КП с реквизитами, сроками и условиями оплаты.

Что здесь нового по сравнению с первым примером:

- **Reference-файлы** — данные отделены от логики. Поменялся прайс — обновил один файл, скилл автоматически берёт новые цены
- **Маршрутизация** — скилл сам решает какой файл подгрузить: для КП — шаблон + прайс + условия, для вопроса по ценам — только прайс
- **Поэтапная загрузка** — в **контекстное окно** попадает только то, что нужно для текущего запроса, а не все файлы разом

Конспект встречи — скилл с командами на компьютере

До сих пор скиллы только генерировали текст. Но скилл может запускать команды на твоём компьютере — **bash**-скрипты, Python, любые утилиты. Это кардинально расширяет возможности: скилл перестаёт быть "умным шаблоном" и становится полноценным инструментом.

Задача: После каждой встречи, созвона или консультации ты тратишь 30-40 минут на конспект: переслушиваешь запись, выписываешь ключевые моменты, оформляешь. Автоматизируем — кидаешь файл записи, получаешь готовый конспект с таймингами.

Что пишешь в skill-creator:

```
/skill-creator
```

Создай скилл для конспекта встречи из видео или аудио записи.

Как работает: я указываю путь к файлу (mp4, mov, mp3, m4a).

Скилл должен:

1. Извлечь аудио из видео (если это видео)
2. Транскрибировать через Whisper (модель large-v3)
3. Сделать структурированный конспект: темы, ключевые решения, задачи с ответственными, тайминги

На моём компьютере установлены ffmpeg и whisper.

Скрипты для извлечения аудио и транскрипции положи в scripts/.

Что создаст skill-creator:

```
meeting-notes/  
├── SKILL.md  
└── scripts/  
    ├── extract-audio.sh  ← извлечение аудио через ffmpeg  
    └── transcribe.sh      ← транскрипция через Whisper
```

Результат — SKILL.md:

```
---  
name: meeting-notes
```

description: "Конспект встречи из видео или аудио. Извлечение аудио через ffmpeg, транскрипция через Whisper, структурированный конспект с таймингами и задачами. Триггеры: конспект встречи, расшифровка записи, транскрипция, meeting notes, протокол совещания, расшифруй запись, конспект созвона, конспект консультации."

Конспект встречи из записи

Отвечай на русском.

When activated

1. Пользователь указал путь к видео/аудио файлу
2. Нужен конспект, транскрипция или протокол встречи
3. Нужно извлечь ключевые моменты из записи

Instructions

Step 1: Определи тип файла

- Видео (mp4, mov, mkv, webm) → нужно извлечь аудио
- Аудио (mp3, m4a, wav, ogg) → сразу на транскрипцию

Step 2: Извлеки аудио (если видео)

Run: `scripts/extract-audio.sh [путь\_к\_файлу]`

→ Создаёт .wav файл рядом с исходным

Step 3: Транскрибируй

Run: `scripts/transcribe.sh [путь\_к\_wav]`

→ Создаёт .txt файл с транскрипцией и таймингами

Step 4: Прочитай транскрипцию и составь конспект

Структура конспекта:

1. ****Участники**** — кто говорил (если можно определить)
2. ****Темы**** — основные темы с таймингами [ММ:SS]
3. ****Ключевые решения**** — что решили, о чём договорились
4. ****Задачи**** — кто, что, к какому сроку
5. ****Важные цитаты**** — дословные фразы, которые важно сохранить

Rules

- НЕ придумывай то, чего не было в записи
- Тайминги — обязательно, с точностью до минуты
- Если качество записи плохое — предупреди, отметь неразборчивые места
- Конспект должен быть полезен тому, кто НЕ был на встрече

Как работает:

Сделай конспект встречи: ~/Downloads/созвон-с-клиентом-15-04.mp4

Скилл запускает `extract-audio.sh` → извлекает аудио → запускает `transcribe.sh`
→ получает текст с таймингами → анализирует → выдаёт:

Участники: Олег (наша сторона), Дмитрий (клиент)

Темы:

[00:00] Обсуждение ТЗ на редизайн сайта

[12:30] Сроки и этапы работы

[24:15] Бюджет и порядок оплаты

Ключевые решения:

- Делаем в два этапа: сначала главная + каталог, потом остальное
- Дизайн-макеты — до 25 апреля

Задачи:

- Олег: отправить бриф до пятницы
- Дмитрий: прислать брендбук и логотип в SVG

Что здесь принципиально нового:

- **Скрипты** — скилл запускает реальные команды на твоём компьютере: `ffmpeg` для аудио, `whisper` для транскрипции. Это не текстовая генерация — это автоматизация рабочего процесса
- **Пайплайн** — несколько шагов последовательно: файл → аудио → текст → конспект. Каждый шаг зависит от предыдущего
- **Работа с файлами** — скилл принимает файл, создаёт промежуточные файлы, выдаёт результат. Всё на твоей машине, ничего не уходит в облако

Скилл выполняет команды на твоём компьютере — это мощная возможность, но и ответственность. Проверяй скрипты перед использованием, особенно если скилл написан не тобой. Подробнее о безопасности — в [главе 18](#).

Отчёт из AmoCRM — скилл с внешним API и ключами

Все предыдущие скиллы работали локально — генерировали текст или обрабатывали файлы на компьютере. Но скилл может обращаться к внешним сервисам через [API](#). Это открывает совершенно другой уровень: CRM, аналитика, маркетплейсы, банки — что угодно, где есть API.

Задача: Ты руководитель отдела продаж. Каждое утро открываешь AmoCRM, руками смотришь просроченные сделки, считаешь кто из менеджеров забил на клиентов. 20 минут на рутину. Автоматизируем — скилл подключается к AmoCRM API и делает отчёт одной командой.

Что пишешь в skill-creator:

```
/skill-creator
```

Создай скилл для отчёта по сделкам из AmoCRM.

Я говорю "отчёт по сделкам" или "что по продажам" — скилл:

1. Запрашивает список сделок через AmoCRM API
2. Находит просроченные (нет активности > 3 дней)
3. Группирует по ответственным менеджерам
4. Выдаёт отчёт: кто сколько сделок ведёт, какие горят, общая сумма в работе

API-запросы положи в scripts/. Токен и субдомен компании хранятся в переменных окружения — создай .env файл в папке скилла:

```
AMOCRM_TOKEN=xxx
```

```
AMOCRM_SUBDOMAIN=mycompany
```

Скрипт должен читать их оттуда. Ключи не должны попадать ни в SKILL.md, ни в скрипты, ни в git — только .env.

Документация AmoCRM API:

https://www.amocrm.ru/developers/content/crm_platform/leads-api

Сходи, посмотри как работает авторизация, какие эндпоинты, какие поля возвращаются.

skill-creator откроет документацию, разберётся в авторизации (Bearer token в заголовке `Authorization`), найдёт нужный эндпоинт (`GET /api/v4/leads`), создаст скрипт и SKILL.md.

Результат — структура:

```
amocrm-report/
├── SKILL.md
├── .env          ← AMOCRM_TOKEN и AMOCRM_SUBDOMAIN (в .gitignore!)
└── scripts/
    └── fetch-leads.sh ← запрос к API, возвращает JSON
```

Как работает:

Что по продажам за эту неделю?

Скилл запускает `scripts/fetch-leads.sh` → скрипт читает токен из `.env` → делает запрос к AmoCRM API → возвращает JSON со сделками → скилл анализирует и выдаёт:

Отчёт по сделкам — 16 апреля

Всего в работе: 23 сделки на 2.4 млн ₽

Просрочены (нет активности > 3 дней): 5 сделок

- Иванов: "Поставка оборудования" (450к) — 6 дней без движения
- Петрова: "Техподдержка на год" (180к) — 4 дня
- Иванов: "Доп. лицензии" (90к) — 5 дней

По менеджерам:

- Иванов: 8 сделок, 2 просрочены
- Петрова: 10 сделок, 1 просрочена
- Сидоров: 5 сделок, всё в порядке

Что здесь принципиально нового:

- **Внешний API** — скилл выходит в интернет и запрашивает данные из CRM-системы
- **Авторизация** — Bearer token для доступа к API. skill-creator сам разберётся как авторизоваться, ты просто даёшь ссылку на документацию
- **Переменные окружения** — токен не в коде, а в `.env` файле. Утёк ключ — сменил в одном месте, скилл продолжает работать

Где брать документацию API? У любого сервиса, который предоставляет API, есть документация — обычно на сайте в разделе "Разработчикам" или "API". AmoCRM, Битрикс24, Яндекс.Метрика, HeadHunter — у всех есть. Гуглишь "[название сервиса] API документация", открываешь, находишь нужные эндпоинты. На первых порах лучше найти документацию самому и скинуть ссылку агенту — так результат будет точнее. Со временем можно просто попросить: "найди API документацию для [сервис]" — агент найдёт сам.

skill-creator — не волшебная кнопка

Важный момент: `/skill-creator` — это ассистент, а не автопилот. Он может накосячить. Типичные ошибки:

- **Всё засунул в один файл.** Прайс, шаблон, условия — всё прямо в SKILL.md на 500 строк. Вместо того чтобы разнести по reference-файлам. Результат — раздутый контекст, медленная работа, дорогие токены.
- **Написал весь SKILL.md и references на русском.** Ты общался по-русски — и он всё сделал на русском. Работать будет, но русский текст жрёт на 30+% больше **токенов** чем английский. На длинном скилле с reference-файлами это ощутимо. Лучше попросить перевести SKILL.md и references на английский, а в инструкциях указать "Отвечай на русском". Триггеры в description — на обоих языках.
- **Сделал description слишком общим.** "Помогает с документами" — бесполезно. Модель не поймёт когда активировать.
- **Не вынес повторяющуюся логику в скрипты.** Каждый раз заново генерирует команды вместо вызова готового скрипта.

Ты прочитал этот гайд — у тебя уже есть экспертиза. Используй её: после того как skill-creator создал скилл, **проверь результат**. Открой SKILL.md, пробежись по чеклисту:

1. Данные отделены от логики? (references, а не всё в одном файле)
2. Description содержит триггеры на двух языках?
3. Есть маршрутизация? (если скилл решает несколько типов задач)
4. Повторяющиеся операции вынесены в скрипты?
5. Rules чёткие — модель не сможет их обойти?

Если что-то не так — скажи skill-creator прямо: "description слишком общий, добавь русские триггеры" или "вынеси прайс в references/price-list.md". Он поправит. Твоя задача — не писать код, а **проверить работу и дать обратную связь**. Как руководитель проверяет работу сотрудника.

Ещё примеры задач для скиллов

Формат SKILL.md мы подробно разобрали в четырёх практиках выше. Вот ещё идеи задач, под которые создают скиллы — для вдохновения:

- **Контент-план для эксперта** — маршрутизация по типу запроса: план на месяц → один алгоритм, конкретный пост → `see references/formats-guide.md`, проверка тона → `see references/tones-checklist.md`. Три задачи — один скилл.
- **Онбординг нового сотрудника** — пошаговый процесс со скриптами: собрать данные → проверить документы (`see references/checklist.md`) → создать профиль (`Run: scripts/create-profile.sh`) → отправить welcome. Каждый шаг — либо reference, либо скрипт.
- **Еженедельный отчёт по продажам** — скилл забирает данные из CRM через скрипт, считает метрики, подставляет в шаблон из `references/report-template.md`.
- **Проверка договоров** — загружаешь договор, скилл проверяет по 20+ категориям рисков из `references/risk-categories.md` и выдаёт заключение.

Каждый из этих скиллов строится по тем же принципам: SKILL.md → маршрутизация → references для данных → scripts для действий.

Не обязательно писать с нуля. Тысячи готовых скиллов уже существуют — бери и используй. А ещё лучше: возьми готовый скилл, который решает похожую задачу, и доработай под себя. Это быстрее, чем создавать с нуля, и ты сразу видишь как опытные авторы оформляют инструкции.

>

Где искать:

- `skills.sh` — 91 000+ скиллов, установка одной командой: `npx skills add`

- [SkillsMP.com](#) — 66 000+ скиллов, скачиваешь ZIP

- [NeuralDeep](#) — скиллы для российских сервисов (Яндекс, Битрикс, 1С)

- [awesome-agent-skills](#) — курированный список лучших

9. Description: 400 символов, которые решают всё

Модель видит десятки скиллов одновременно. Какой активировать — она решает **только по description**. Если описание размытое, скилл не вызовется. Если слишком узкое — сработает в одном случае из десяти.

Формула: ЧТО + КОГДА + ТРИГГЕРЫ

```
# Никуда не годится — что делает? когда активировать?
description: "Helps with servers"

# Уже лучше — понятно ЧТО
description: "Full-cycle server management. Deploy, monitor,
  backup, update any software on VPS servers."

# Хорошо — ЧТО + КОГДА + триггеры на двух языках
description: "Full-cycle server management. Deploy, monitor,
  backup, update any software on VPS servers.
  EN triggers: server, deploy, Docker, VPS, domain, DNS, SSL.
  RU triggers: сервер, поставь, подними, задеплой, бэкап."
```

Двуязычные триггеры — наш лайфхак

Если аудитория говорит по-русски, а description только на английском — скилл молчит на "поставь n8n". Модель формально знает русский, но сопоставление description с запросом работает лучше, когда языки совпадают.

Мы тестировали: английские триггеры + русский запрос = активация ~40%. Добавили русские триггеры = ~95%+. Разница огромная.

Хорошие и плохие имена

Хорошее	Плохое	Почему
<code>content-plan</code>	<code>helper</code>	Неинформативно, непонятно что делает
<code>sales-report</code>	<code>my_cool_skill</code>	Подчёркивания запрещены, ни о чём не говорит
<code>data-analysis</code>	<code>-data</code>	Нельзя начинать с дефиса
<code>processing-pdfs</code>	<code>PDF-Processing</code>	Нельзя заглавные буквы

Имя = `kebab-case`. Строчные, дефисы, 2-4 слова. Должно совпадать с именем папки.

Опциональные поля

Кроме `name` и `description` есть:

- `license` — лицензия (MIT, Apache-2.0)
- `compatibility` — требования к окружению
- `metadata` — автор, версия, теги
- `allowed-tools` — список инструментов, которые скилл может использовать без запроса разрешения. Например, `allowed-tools: ["Bash", "Read", "Write"]` — скилл сам читает файлы и выполняет команды, не спрашивая "можно?" на каждый вызов. Для DevOps-скилла это критично — иначе установка 6 сервисов превратится в 200 попапов подтверждения.

10. Инструкции: как разговаривать с моделью

Не учи — направляй

Модель знает как писать SQL, Docker Compose, Nginx-конфиг, составить SWOT-анализ. Не трать токены на учебник. Пиши только:

- **Твой** порядок действий (что за чем)
- **Твои** параметры (конкретные порты, имена, форматы)
- **Твои** правила (чего нельзя делать)

Лайфхак: пиши на русском, потом переводи

На этапе создания и тестирования — пиши SKILL.md на русском. Так проще думать, быстрее итерировать, легче замечать ошибки в логике. Когда скилл стабилизировался — попроси агента перевести на английский. Инструкции на английском расходуют примерно на 30% меньше токенов, чем на русском (кириллица = больше токенов на символ). На длинных сессиях это ощутимая разница.

Триггеры в description при этом оставь двуязычными — они нужны для активации.

Давай столько свободы, сколько нужно задаче

Жёсткий контроль (DevOps, финансы, юридические документы): каждый шаг прописан, порядок фиксирован.

Порядок установки (СТРОГО):

1. setup-server.sh → Docker, UFW
 2. install-caddy.sh → reverse proxy
 3. cloudflare-dns.sh → DNS-записи
- НЕ менять порядок. НЕ пропускать шаги.

Гибкость (отчёты, анализ): шаблон + свобода адаптации.

Используй шаблон из assets/report-template.md.
Если данных по разделу нет — пропусти его.

Свобода (написание текстов, генерация идей): только тема и ограничения.

Напиши статью на тему X.
Тон: разговорный, с юмором. Длина: 1500-2000 слов.

Правильный уровень абстракции

Слишком расплывчато (модель не знает что делать):
"Настрой сервер"

Слишком детально (сломается при любом отклонении):

"Если порт 5678 занят, убей процесс с PID..."

Правильно (направляет, но не ломается):

"Проверь что Docker установлен и версия ≥ 24 .

Если нет — установи. Если порт занят — покажи пользователю что его занимает и спроси как поступить."

Собирай данные пошагово

Не вываливай на пользователя 10 вопросов разом. Спрашивай по одному.

Плохо:

"Дай IP, пароль root, email, домен, API token..."

Хорошо:

Step 1: "Есть VPS? Скинь IP и пароль root."

→ Если нет: "Купи VPS. Ubuntu 22.04. Вернись с IP."

Step 2: "Какой email?"

Step 3: "Есть домен?" → Да / Нет / Что это?

Паттерны воркфлоу

Не изобретай колесо — большинство скиллов укладываются в один из трёх шаблонов:

Чеклист — для задач с фиксированным набором шагов:

Pre-deploy checklist

- [] SSH доступ работает
- [] CF token валидный
- [] Домен активен в Cloudflare
- [] Ресурсы достаточны (RAM, disk)
- [] План показан пользователю и подтверждён

Цикл валидации — для итеративных задач (генерация текста, анализ):

Validation Loop

1. Выполни действие
2. Проверь результат по критериям

3. Если ошибка → исправь и вернись к шагу 1
4. Если всё ок → перейди к следующему этапу

План → Подтверждение → Выполнение → Проверка — для сложных многошаговых задач:

Workflow

1. PLAN: Собери данные, покажи план
2. VALIDATE: Подтверди с пользователем
3. EXECUTE: Выполняй по шагам
4. VERIFY: Проверь что всё работает
5. REPORT: Сгенерируй отчёт

Наш devops-ак использует именно третий паттерн: 6 фаз от сбора данных до PDF-отчёта. Ни один сервис не ставится без "Correct? Starting?"

Не забывай про Examples

Примеры входов/выходов — самая недооценённая часть скилла. Модель по ним понимает, какой формат ты ожидаешь. Без примеров — каждый раз разный результат. С примерами — стабильный.

Минимум два: один типичный запрос, один граничный случай.

Examples

Типичный запрос

User: "Создай отчёт по продажам за 24-30 марта"

Result:

Еженедельный отчёт | 24-30 марта 2026

Summary

Выручка: 2 450 000 руб (+12% к прошлой неделе)

Конверсия: 3.2% (стабильно)

Граничный случай

User: "Создай отчёт" (без указания периода)

Result:

Уточняю: за какой период сделать отчёт?

Последняя неделя / Последний месяц / Другой период?

11. Тестируй, ломай, чини

Первая версия скилла — это черновик. Рабочий скилл — это 3-5 итераций. Не пытайся сделать идеально с первого раза. Задача — быстро найти что сломано и починить.

Три вещи, которые нужно проверить

1. Триггерится ли скилл?

Самая частая проблема — скилл создан, но не активируется. Или активируется когда не нужно. Проверяем на примере скилла review-reply (ответы на отзывы):

Запрос	Должен сработать?	Результат
"Ответь на отзыв клиента"	Да	
"Вот отзыв: отличный кофе, приду ещё"	Да	
"Напиши ответ на негативный отзыв в 2ГИС"	Да	
"Напиши стихотворение про кофе"	Нет	
"Какая погода завтра"	Нет	

Запусти 5-6 таких запросов. Если скилл молчит на "ответь на отзыв" — description слишком узкий. Если срабатывает на "напиши стихотворение" — слишком общий.

2. Правильный ли результат?

Скилл триггернулся — но выдал что нужно? Чеклист:

- **Формат правильный?** КП содержит все разделы из шаблона? Конспект имеет тайминги?
- **Данные корректные?** Цены из прайса, а не выдуманные? Имя клиента правильное?
- **Тон правильный?** Ответ на отзыв — тёплый и конкретный, а не "Уважаемый клиент, благодарим за обращение"?
- **Маршрутизация работает?** Попроси "покажи прайс" — скилл открыл `price-list.md`, а не шаблон КП?

3. Что происходит когда всё идёт не так?

Самый важный тест — на граничных случаях:

- Не указал название компании → скилл спрашивает, а не выдумывает?
- Запросил услугу, которой нет в прайсе → скилл говорит "нет в прайсе", а не подставляет цену от балды?

- Кинул битый аудио файл → скилл предупреждает о проблеме, а не падает молча?
- Запустил скилл второй раз подряд → работает стабильно, не "забыл" предыдущий контекст?

Типичные проблемы и решения

Проблема	Что значит	Что делать
Скилл не триггерится	Description слишком узкий	Добавить синонимы, русские триггеры
Триггерится когда не нужно	Description слишком общий	Уточнить формулировку, убрать общие слова
Результат каждый раз разный	Нет примеров в инструкциях	Добавить секцию Examples с 2+ образцами
Игнорирует часть инструкций	SKILL.md слишком длинный	Вынести данные в references/
Делает лишнее	Нет раздела Rules	Добавить жёсткие ограничения
Жрёт много токенов	Всё в одном файле	Разнести по references/
Скрипт падает	Нет обработки ошибок	Добавить проверки в скрипт

Как итерировать

Не нужно "думать что может сломаться". Нужно **ломать и смотреть что сломалось**:

1. Запусти скилл 5 раз с разными запросами — нормальными и странными
2. Запиши что сломалось (не триггернулся? плохой результат? неправильный формат?)
3. Открой SKILL.md → найди причину → поправь (или попроси skill-creator поправить: "description не ловит запрос 'что по отзывам', добавь этот триггер")
4. Запусти ещё 5 раз
5. Повтори пока не перестанет ломаться

Обычно хватает 3-5 итераций. После первой — скилл работает на 60%. После третьей — на 90%. Оставшиеся 10% ловятся в реальной работе. Это

нормально — даже production-скилл devops-ак мы дорабатываем после каждого инцидента.

Часть IV — Как мы сделали devops-ак

Теперь, когда ты понимаешь как устроены скиллы и как их создавать — разберём наш production-скилл по косточкам. Тот самый, который заменил 8 агентов.

12. Файловая структура

```
devops-ak/
├── SKILL.md                # Точка входа. 91 строка.
├── references/             # Подробные инструкции (10 файлов)
│   ├── onboarding-guide.md # Пошаговый сбор данных у пользователя
│   ├── ops-commands.md     # Статус, бэкапы, обновления
│   ├── troubleshooting.md  # Решение типичных проблем
│   ├── vps-guide.md        # Как купить VPS, подключиться по SSH
│   ├── cloudflare-setup.md # Настройка Cloudflare и DNS
│   ├── ssh-security.md     # SSH-ключи и безопасность
│   ├── github-deploy.md    # Деплой проектов с GitHub
│   ├── post-install.md     # Действия после установки
│   ├── scripts-reference.md # Описание всех скриптов и параметров
│   └── report-template.md  # Шаблон PDF-отчёта
├── scripts/               # Исполняемые скрипты (15 файлов)
│   ├── setup-server.sh     # Docker, UFW, пользователи
│   ├── install-caddy.sh    # Reverse проху, сеть infra
│   ├── cloudflare-dns.sh   # А-записи в Cloudflare
│   ├── install-n8n.sh       # n8n: compose + admin
│   ├── install-supabase.sh  # Supabase: полный стек
│   ├── install-openclaw.sh  # Автономный агент
│   ├── install-lightrag.sh  # База знаний
│   ├── backup.sh           # Бэкап всех данных
│   ├── restore.sh          # Восстановление
│   ├── generate-report.py   # Генерация PDF
│   └── ...
└── assets/                # Шаблоны
```

Это скилл уровня 3: SKILL.md + 10 references + 15 скриптов. Маршрутизатор в SKILL.md (91 строка) направляет к нужному reference. Скрипты вызываются из инструкций.

Когда пользователь просит "проверь сервер" — модель читает только `ops-commands.md` (~40 строк). Остальные 9 файлов не трогает. В контексте ~2000 токенов вместо ~8000.

13. SKILL.md строка за строкой

Метаданные

```
---
name: devops-ak
description: "Full-cycle server and infrastructure management...
  EN triggers: server, deploy, install, VPS, Docker...
  RU triggers: сервер, поставь, подними, установи, задеплой..."
---
```

~80 триггерных слов на двух языках. Двухязычные триггеры критичны — почему, разобрано в [главе 9](#).

Маршрутизация

```
## Route by request type
1. Deploy / install → Deploy Workflow below
2. Server status / check → see references/ops-commands.md
3. Backup / restore → see references/ops-commands.md
4. Fix / troubleshoot → see references/troubleshooting.md
5. VPS purchase / SSH → see references/vps-guide.md
6. Cloudflare setup → see references/cloudflare-setup.md
7. SSH keys → see references/ssh-security.md
8. GitHub project deploy → see references/github-deploy.md
9. Existing server → scan → show to user → ask
```

Строка 9 — результат инцидента. Скилл молча установил софт на сервер, где уже что-то работало. Снёс чужой Portainer с данными. Теперь: если сервер не пустой — сначала покажи что на нём есть и спроси.

Deploy Workflow — 6 фаз

Phase 1: Collect Data → пошаговый сбор в `onboarding-guide.md`. Ранняя версия спрашивала всё залпом — пользователи уходили. Переписали на пошагово.

Phase 2: Verify → SSH, Cloudflare token, домен активен, ресурсы. Раньше не было — Caddy падал без DNS, Cloudflare-токен невалидный, сервер в полусобранном состоянии.

Phase 3: Confirm → показать план, спросить "Correct? Starting?". Ни один сервис не ставится без "да".

Phase 4: Install → строгий порядок: Docker → Caddy → DNS → сервисы → LightRAG последним. Каждое правило порядка — результат конкретного бага (Caddy создаёт сеть `infra`, LightRAG настраивает MCP к другим сервисам).

Phase 5: Verify → `docker ps`, `curl` каждый URL, `ufw status`. Конкретные проверки, не "вроде поставилось".

Phase 6: Report → PDF с логинами + JSON-конфиг для следующего запуска.

Rules

- NEVER delete data without user confirmation
- Suggest backup BEFORE updates
- Do NOT restart services during backup
- SSH keys are OPTIONAL — do not push on user

Каждое правило — результат реального инцидента. "NEVER delete" — модель удалила контейнер с volumes при "обновлении". "Backup BEFORE" — обновление Supabase сломало базу без отката. "SSH keys OPTIONAL" — модель заставляла новичка настраивать SSH-ключи, хотя он не знал что это.

Security

- UFW: only 22, 80, 443
- Service ports NEVER exposed externally
- Credentials: openssl rand -hex 16 minimum
- server-config.json: chmod 600, .gitignore

Безопасность зашита в инструкции. Модель физически не может открыть порт 5432 наружу.

14. Скрипты: когда модели нужны руки

Большинству скиллов скрипты не нужны. Если скилл пишет тексты, анализирует данные, составляет отчёты — хватит одного SKILL.md.

Скрипты нужны когда:

- Сложная bash-логика (установка софта, конфигурация)
- Нужна **идемпотентность** (безопасно запустить повторно)
- Нужна предсказуемость (одни параметры → один результат)
- Обращение к внешним API (Supabase, системы перевозчиков, CRM)

В devops-ак каждый скрипт следует четырём правилам:

1. **Идемпотентность.** `install-n8n.sh` — если контейнер уже работает, не трогает. Безопасно запустить 5 раз.
2. **Генерация credentials.** Пароли через `openssl rand -hex 16`, не "Password123".
3. **JSON-лог.** Результат в `/root/logs/` — скилл читает для отчёта.
4. **Exit code.** 0 = успех, остальное = ошибка.

Почему скрипты, а не команды от модели?

Мы начинали без скриптов. Модель сама генерировала Docker Compose. Проблемы:

- Каждый раз compose чуть отличался (модель "творчески" подходила)
- Забывала healthcheck или restart policy
- Пароли вида "n8n_password_2024"
- Повторный запуск ломал предыдущую установку

С готовыми скриптами: один compose, протестирован 50 раз, пароли криптостойкие. Модель вызывает — скрипт делает.

Правило: текстовый скилл → скрипты не нужны. Скилл работает с системой → напиши скрипты для критичных операций.

Часть V — На продакшн

15. От прототипа к стабильной системе

Generic vs конкретный скилл

Первая версия — **generic**: модель сама генерирует SQL, сама пишет конфиги. Быстро для прототипа.

```
# Generic: модель сама пишет SQL каждый раз
"Получи данные о водителях из Supabase"
→ Модель генерирует: SELECT * FROM drivers WHERE status = 'active'
```

Проблемы: токены на генерацию каждый раз, SQL чуть отличается, **UPDATE** без **WHERE** = катастрофа.

Продакшн — конкретный: SQL зашит в скрипт, параметры протестированы.

```
# Конкретный: SQL зашит, проверен, стабилен
get-driver-status(telegram_id: "12345")
→ { driver_id: "uuid", status: "active", name: "Иванов" }
```

Правило: generic для прототипа, конкретный для продакшна.

Что отличает production-скилл

Прототип	Продакшн
Один SKILL.md на 300 строк	SKILL.md 80 строк + references
Нет Examples	2+ примера входов/выводов
Нет Troubleshooting	Обработка 5+ типичных ошибок
Нет Rules	Жёсткие ограничения
Модель сама решает	Скрипты для критичных операций
"Работает на моей машине"	Тестировано на 5+ запросах

16. Автоматизация: чтобы скилл работал без тебя

Уровень 1: Ручной вызов

`/devops-ak` проверь сервер" → результат

Уровень 2: Scheduled Tasks (Claude Code)

"Каждые 6 часов проверяй статус сервера"

Ограничение: работает пока Claude Code открыт

Уровень 3: Автономный агент (OpenClaw)

24/7 на VPS, крон-задачи, результат в Telegram

Работает без тебя — даже ночью, в отпуске

Scheduled Tasks в Claude Code

Прямо в чате:

"Создай scheduled task: каждые 6 часов запускай

`/devops-ak` проверь статус сервера 185.x.x.x"

Под капотом — cron-задача. Удобно для: утренние сводки, проверки здоровья, еженедельные отчёты.

Ограничение: закрыл приложение — задачи не выполняются.

OpenClaw: 24/7 без тебя

Автономный агент на сервере. Тот же формат SKILL.md, но работает круглосуточно, общается через Telegram. Чтобы скилл запускался автоматически — ставишь его на крон, как и в Claude Code:

В Telegram-чате с агентом OpenClaw:

"Поставь на крон: каждый день в 8:00 запускай

`/morning-digest`"

Или добавь задачу через Dashboard OpenClaw

Агент создаёт крон-задачу, и каждый день в 8:00 запускает скилл → собирает данные → формирует сводку → шлёт тебе в Telegram. Ты просыпаешься — сводка уже в чате.

Совет: после настройки крона проверь через Dashboard OpenClaw, что задача создана и следующий запуск запланирован. Или вызови скилл вручную разок прямо в чате — убедись что работает, потом уже на автомат.

Не прыгай через ступени

Не ставь скилл на крон, пока он не работает руками. Сначала 10 ручных запусков, потом автоматизация.

17. Скиллы на стероидах: воркфлоу, цепочки, оркестрация

Скилл как полноценный процесс

Один скилл может оркестрировать сложный процесс с несколькими внешними системами:

Утренняя аналитическая сводка:

```
---
name: morning-digest
description: "Утренняя сводка: данные из CRM, аналитика,
  прогноз, отправка в Telegram."
---

## Workflow
1. Забери данные за вчера из Supabase (MCP: supabase-mcp)
2. Найди паттерны в LightRAG (MCP: lightrag)
3. Сформируй отчёт: выручка, конверсия, топ-3, прогноз
4. Сохрани прогноз в Supabase
5. Отправь сводку в Telegram
```

Обработка входящей заявки:

```
---
name: lead-processor
```

```
description: "Обработка входящих заявок: валидация,  
создание в CRM, уведомление менеджера."
```

```
---
```

```
## Workflow
```

1. Получи данные заявки (имя, телефон, запрос)
2. Проверь дубли в Supabase
3. Если клиент существует → обновь
4. Если новый → создай, найди похожие кейсы в LightRAG,
отправь менеджеру в Telegram с контекстом

Цепочки скиллов

Один скилл может вызвать другой:

```
## After deployment
```

```
Run `/monitoring-setup` to configure health checks.
```

Или последовательность: исследование → отчёт → презентация.

Когда один скилл, а когда цепочка

Большинство задач решается одним скиллом. Цепочка нужна когда задача чётко делится на независимые этапы, или разные этапы требуют разных инструментов.

Скиллы и MCP — разные подходы к одной задаче

MCP и скиллы могут решать похожие задачи, но делают это по-разному.

MCP-сервер — набор инструментов с широкими возможностями. Подключил MCP для Supabase — агент может выполнять любые SQL-запросы, создавать таблицы, управлять пользователями. Всё что угодно, на лету. Но при подключении загружаются схемы VCEX инструментов, даже если нужен один.

Скилл — конкретные инструкции к конкретному процессу. Может обращаться к тем же API через скрипты, но вызывает конкретный запрос с конкретными параметрами. Загружается только когда нужен.

	MCP	Skill
Подход	Широкий доступ ко всей системе	Точные инструкции для конкретной задачи
Контекст	Загружает схемы всех инструментов сразу	Загружается по запросу, поэтапно
Гибкость	Любой запрос на лету	Только то, что прописано
Предсказуемость	Агент сам решает как действовать	Протестированные шаги и скрипты
Пример	<code>run_sql("SELECT * FROM ...")</code> — агент формирует SQL каждый раз	<code>get-weekly-report.sh</code> — готовый SQL, проверен 50 раз
Лучше для	Разработка, исследование, ad-hoc задачи	Продакшн-процессы, регулярные задачи

Вместе: скилл описывает процесс, а для шагов, где нужна гибкость — использует MCP-инструменты. Не конкуренты, а разные уровни.

Совет: начни с одиночных скиллов. К цепочкам переходи когда упрёшься в ограничения.

18. Безопасность: чужие скиллы и свои грабли

Скилл — это инструкции с доступом к твоей системе. Вредоносный скилл может прочитать SSH-ключи, `.env`-файлы, выполнить произвольный bash.

Проверка перед установкой

1. **Читай SKILL.md** — 2 минуты. Мутные инструкции = красный флаг.
2. **Проверяй scripts/** — `curl | sh` из незнакомого URL = нет.
3. **Смотри references/** — внешние URL вместо локальных файлов → данные уходят наружу.
4. **Проверь автора** — GitHub-профиль, звёзды, другие проекты.

```
# Безопасная установка:
npx skills add owner/repo --dry-run # скачай, НЕ устанавливай
```

```
cat SKILL.md && cat scripts/*      # прочитай ВСЕ файлы  
cp -r skill-name ~/.claude/skills/  # если чисто — устанавливай
```

Лучший скилл — написанный тобой. Готовый скилл — стартовая точка, но всегда проверяй.

Справочник

Шаблон SKILL.md

```
---
name: [kebab-case, 2-4 слова, совпадает с папкой]
description: "[ЧТО делает. КОГДА использовать.
  Триггеры: слово1, слово2, слово3.
  200-400 символов.]"
---
```

```
# [Название]
```

```
Respond in the user's language.
```

```
## When activated
[2-4 ситуации]
```

```
## Instructions
```

```
### Step 1: [Понять запрос]
### Step 2: [Собрать данные]
### Step 3: [Выполнить]
### Step 4: [Оформить результат]
```

```
## Examples
```

```
### Типичный запрос
User: "[фраза]"
Result: [3-5 строк]
```

```
### Граничный случай
User: "[необычный запрос]"
Result: [3-5 строк]
```

```
## Troubleshooting
```

```
Error: [Проблема]
Solution: [Решение]
```

```
## Rules (ALWAYS follow)
```

- [Ограничение 1]
- [Ограничение 2]

Если скилл вырос за ~3-4 страницы — выноси в `references/`.

Чеклист готовности

Перед использованием:

- ☐ `name` = имя папки, kebab-case
- ☐ `description`: ЧТО + КОГДА + триггеры (на языке аудитории)
- ☐ SKILL.md < 5000 токенов (детали — в `references/`)
- ☐ Есть секция When activated
- ☐ Пошаговые инструкции (3+ шагов)
- ☐ Примеры входов/выходов (2+ штуки)
- ☐ Troubleshooting (2+ сценария)
- ☐ Rules с жёсткими ограничениями

Перед продакшном:

- ☐ Активируется на 5+ формулировках
- ☐ НЕ активируется на нерелевантных
- ☐ Результат стабилен при повторях
- ☐ Граничные случаи обработаны
- ☐ Идемпотентно
- ☐ Критичные операции — через скрипты

Словарь терминов

Agent Skill — набор инструкций в формате SKILL.md, обучающий AI-агента конкретной задаче. По сути — структурированный рецепт для ИИ.

API (Application Programming Interface) — интерфейс, через который программы общаются друг с другом. Например, скрипт отправляет запрос к API Supabase и получает данные из базы. Как "окошко выдачи" в фастфуде — ты говоришь что нужно, получаешь результат.

Bash — командная оболочка (терминал) в Linux/macOS. Скрипты на Bash — это текстовые файлы с командами, которые выполняются последовательно. Как автоматический набор действий в терминале.

CLAUDE.md — файл с правилами проекта, который агент читает при каждом запуске. В отличие от SKILL.md, загружается всегда, а не по запросу.

Credentials — учётные данные: логины, пароли, API-ключи, токены доступа. Всё, что нужно для авторизации в сервисе.

DNS (Domain Name System) — система, которая превращает доменное имя (example.com) в IP-адрес сервера (185.x.x.x). Как телефонная книга интернета.

Docker — платформа для запуска приложений в изолированных контейнерах. Каждый сервис (база данных, веб-сервер) работает в своём контейнере, не мешая другим.

Docker Compose — инструмент для запуска нескольких Docker-контейнеров одной командой. Описываешь все сервисы в одном файле `docker-compose.yml` и поднимаешь разом.

Generic скилл — скилл, который даёт агенту широкую свободу действий (сам пишет SQL, сам решает как подступиться). Противоположность конкретного скилла, где действия зашиты в скрипты.

Git / GitHub — система контроля версий (Git) и платформа для хранения кода (GitHub). Позволяет отслеживать изменения, работать в команде, хранить проекты.

JSON (JavaScript Object Notation) — формат хранения данных в виде пар "ключ: значение". Читается и людьми, и программами. Пример: `{"name": "Иван", "age": 30}`.

Kebab-case — стиль именования, где слова пишутся строчными и разделяются дефисами: `my-skill-name`. Стандарт для имён скиллов.

Markdown — простой язык разметки текста. `#` — заголовок, **жирный**, `- список`. SKILL.md написан на Markdown.

Маршрутизация — секция в SKILL.md, которая направляет разные типы запросов к разным файлам. Как оглавление книги: "если про бэкапы — читай главу 3".

MCP (Model Context Protocol) — протокол подключения внешних систем (баз данных, API, SaaS-сервисов) к AI-агенту. Как USB-разъём: стандартный способ подключить что угодно.

Контекстное окно — "оперативная память" модели. Максимальный объём текста (в токенах), который модель может обработать за раз. Включает инструкции, диалог и результаты работы.

Крон (cron) — планировщик задач в Unix/Linux. Позволяет запускать команды по расписанию: каждый час, каждый день в 8:00, каждый понедельник. Используется для автоматизации скиллов.

Идемпотентность — свойство операции, которую можно выполнить повторно без нежелательных последствий. Запустил скрипт 5 раз подряд — результат тот же, что после первого раза.

OpenClaw — автономный AI-агент, который работает на сервере 24/7 и общается через Telegram. Поддерживает тот же формат скиллов (SKILL.md).

Оркестрация — координация работы нескольких скиллов или агентов для решения сложной задачи. Как дирижёр: каждый инструмент играет своё, а вместе — симфония.

Поэтапная загрузка — принцип, при котором инструкции загружаются не все сразу, а по мере необходимости. Сначала метаданные, потом SKILL.md, потом нужный reference-файл.

Промпт — текстовый запрос к AI-модели. "Напиши письмо клиенту" — это промпт. Системный промпт — начальная инструкция, определяющая поведение модели.

Scheduled Tasks — запланированные задачи в Claude Code. Скилл запускается автоматически по расписанию, без участия пользователя.

Скрипт (script) — текстовый файл с набором команд, которые выполняются автоматически. Скрипт `install-n8n.sh` — это последовательность bash-команд для установки n8n.

Слэш-команда — команда, начинающаяся с `/`. Ввод `/devops-ak` в чате гарантированно запускает скилл devops-ak. Способ явного запуска скилла.

SSH (Secure Shell) — протокол безопасного удалённого подключения к серверу. Через SSH ты управляешь сервером из своего терминала, как если бы сидел за ним.

SSL — сертификат безопасности для сайта. Обеспечивает шифрование (https:// вместо http://). Без SSL — браузер показывает "небезопасно".

Токен — единица текста для LLM. Примерно ~4 символа английского текста или ~1-2 символа русского. Чем больше токенов занимают инструкции — тем меньше места для работы.

TodoWrite — встроенный инструмент Claude Code для отслеживания задач. Модель создаёт чеклист и отмечает выполненные шаги — удобно для длинных многошаговых процессов, где легко потерять прогресс.

Триггер — слово или фраза в описании скилла, по которой модель решает его активировать. Пользователь пишет "подними сервер" → модель видит триггер "подними" → запускает скилл.

Train/test split — разделение данных на обучающую и тестовую выборки. В контексте skill-creator: 60% тест-кейсов используются для оптимизации description, 40% — для проверки результата.

Файрвол (UFW) — программа, которая контролирует сетевой доступ к серверу. Разрешает подключения только по определённым портам (например, 22 для SSH, 80/443 для веб).

VPS (Virtual Private Server) — виртуальный сервер в облаке. Арендуешь у хостера — получаешь полноценный компьютер с Linux, доступный 24/7 по SSH.

Воркфлоу (workflow) — последовательность шагов для выполнения задачи. Скилл по сути и есть описание воркфлоу: шаг 1, шаг 2, проверка, результат.

Ссылки и ресурсы

Документация

- [Agent Skills](#) — спецификация стандарта
- [Anthropic: Skills для Claude Code](#)
- [Anthropic: Context Engineering](#)

Маркетплейсы скиллов

- [skills.sh](#) — 91 000+ скиллов, установка через `npx skills add`
- [SkillsMP.com](#) — 66 000+ скиллов, браузер + ZIP
- [awesome-agent-skills](#) — курированный список
- [NeuralDeep](#) — скиллы для российских сервисов (Яндекс, Битрикс, 1C)

Примеры скиллов

- [anthropics/skills](#) — официальные скиллы Anthropic
- [cc-devops-skills](#) — 31+ DevOps-скилл
- [ComposioHQ / awesome-claude-skills](#) — коллекция скиллов
- [claude-legal-skill](#) — юридический скилл, 41 категория рисков

Что дальше

Скиллы — один инструмент. Когда задача перерастает один агент (изоляция прав, разные модели, разные пользователи) — нужна мультиагентная архитектура.

Подробнее — в **"Мультиагентные AI-системы: практический гайд"** в приватном канале:

- Когда хватит одного агента + скиллов, а когда нужно разделять
- Архитектурные паттерны: Router, Pipeline, Fan-out, Orchestrator
- Безопасность: prompt injection, изоляция прав
- Выбор моделей под задачи и бюджеты

Оба гайда дополняют друг друга: этот учит создавать скиллы, тот — собирать из них системы.

Источники

1. [Anthropic — Building Effective Agents](#)
 2. [Anthropic — Extend Claude with Skills](#)
 3. [Agent Skills — спецификация стандарта](#)
 4. [Anthropic — Context Engineering](#)
 5. [MCP — Model Context Protocol](#)
 6. [Intuition Labs — Skills vs MCP](#)
 7. [Ravikanth Chaganti — Agent Skills vs MCP](#)
-

Материал из канала [Олег Лаврентьев](#)

Апрель 2026